GigBid: R&D report

AI-4016 R&D assignment, design review. Repo `github.com/awwab-03/gigbid`, live `https://40-233-123-21.sslip.io`. Every number cites a file in `evals/results/` or a commit.

1. Problem and constraints

A freelancer spends hours a week reading job posts, discarding scams and poor fits, pricing, and writing proposals that prove relevant experience. GigBid automates the reading, judging and drafting and keeps the human on every action that has consequences.

Never automated: submitting a proposal, spending Connects, sending a client message, accepting a contract. Each is an `interrupt()` in a LangGraph graph (`agent/graphs/bid_graph.py:79`, `conversation_graph.py:49`), the autonomy setting has no "auto-send" level (`agent/config.yaml: autonomy`), and live mode never scrapes or uses browser automation (`mcp_servers/marketplace/adapters/upwork_live.py`).

Declared budgets (`agent/config.yaml: budgets`): p95 latency 25 s per job analysed and 90 s per proposal; cost USD 0.004 per job and 0.02 per proposal; at most 40 graph steps, 2 Critic revisions, 60k tokens per job. We first assumed Cerebras was free; it was not. All 11 Nov runs (RAG, decisions, model comparison) used **USD 5.27** of real usage, paid by a USD 5 promotional credit (Cerebras billing page: qwen-3.8-27b 5.09, gpt-oss-120b 0.18). When the credit ran out, the remaining experiments moved to open-weights **Gemma 4** (26B-A4B small, 31B large) on Google AI Studio's free tier (no billing account, USD 0). Because the per-run split of the Cerebras bill is not available, cost is reported as tokens per job / proposal plus that total.

Data. No simulator or labelled data was supplied, so we built GigSim and a synthetic, seeded (seed 7) world: 3 personas with portfolios, 840 jobs, 600 with gold labels (bid/skip and reasons, scam type, planted injections, gold evidence, fair price band). `docs/DATA_CARD.md` documents generation and its limits; the most important limit is that GigSim's simulated client is a rule-based model we wrote, so "reply rate" measures agreement with those rules, not with real clients.

2. Architecture

Bid graph (one thread per job; rendered from code in `docs/images/bid_graph.mmd`): `intake` fans out with `Send` to **Job Analyst**, **Risk Assessor** and **Fit Matcher** in parallel; they join at `decide` (deterministic rules), then `pricing` (deterministic quote + Connects expected value), `writer`, `verifier` (claim verification), `critic` (at most 2 revisions), `compliance`, `approval` (`interrupt`) and `submit` (idempotent). **Conversation graph** (one thread per client thread): classify intent, find calendar slots, draft reply, compliance, approval, send.

Agent	Why it exists separately	Model
Job Analyst	turns an untrusted post into a typed `JobSpec` the rest can rely on	small (heuristic fallback)
Risk Assessor	scam and injection checks must run before anything is drafted; rules first, model only when ambiguous	small, rarely
Fit Matcher	requirement-to-evidence matrix is retrieval, not generation	none (Portfolio MCP)
decide / pricing	deterministic and testable (ADR 0005)	none
Writer	grounded drafting with `[E#]` citations	large
Verifier	removes unsupported claims before the Critic sees them	cross-encoder
Critic	independent quality review, bounded loop	large
Compliance Guard	platform rules, contact details, guarantees, policy version	small + rules

Marketplace port. Agents never call GigSim directly: the Marketplace MCP server has a `MarketplaceBackend` port with a GigSim adapter and an Upwork-MCP adapter (`mcp_servers/marketplace/adapters/`). **MCP surface:** 22 tools on three servers (`docs/TOOLS.md`), per-agent allowlists (`agent/config.yaml: tool_allowlist`), bearer service tokens, write tools that require idempotency keys and support `dry_run`, a hash-chained audit log per server, schema snapshot tests. **Stores:** Postgres for LangGraph checkpoints, the LangGraph store (style and outcome memory) and app tables; pgvector for portfolio vectors. **Deployment** (ADR 0008): one Oracle Always Free VM (1 GB), MCP servers in-process, inline worker and scheduler, Caddy HTTPS.

3. R&D question A: does tool granularity change agent behaviour?

Hypothesis. A few coarse tools with an `action` enum lower tool-selection errors (fewer choices) but raise argument errors (one schema for many intents); rewriting our descriptions with "use for / not for" guidance helps most on the ambiguous pairs (search vs match vs verify, client-history facets).

Setup. 40 tasks with a known correct first call (`evals/runners/tool_granularity.py`), four surfaces: *fine* (42 narrow tools), *coarse* (3 tools with `action`), *ours* (the real 22 MCP tools, schemas read from the servers) and *ours_rw* (same tools, rewritten descriptions). Two open models, Gemma 4 31B and Gemma 4 26B-A4B, 0 provider errors. Scored on the first call: tool-selection accuracy, argument accuracy on key arguments, tokens per task.

Surface (tools)	31B tool / args	26B tool / args	Tokens per task
fine (42)	0.950 / 0.900	0.925 / 0.900	1,917
coarse (3)	0.925 / 0.700	0.950 / 0.600	486
ours (22)	0.975 / 0.975	0.925 / 0.925	3,421
ours_rw (22)	**0.975 / 0.975**	**0.975 / 0.975**	3,614

Source: `evals/results/tool_granularity.json`, 40 tasks per cell; one task is 0.025.

Finding. The hypothesis holds for arguments, not for selection. Collapsing to 3 tools did not make choosing easier (0.925 to 0.950, within one or two tasks of the others) but cost 20 to 30 points of argument accuracy: with one schema for many intents, the model fills the wrong fields for the chosen `action`. The rewritten descriptions helped the smaller model most (0.925 to 0.975 on both scores) and changed nothing for the larger one. The cost is tokens: our schemas are 7 times the coarse surface per call. **Decision:** keep the 22 tools with rewritten descriptions. With 40 tasks, differences under about 0.05 are not distinguishable from noise.

4. R&D question B: RAG variants, claim verification, what ships

Hypothesis (assignment form). Requirement decomposition plus hybrid retrieval raises evidence Recall@5 by at least 15 points over V0 on jobs with four or more requirements.

Retrieval (450 dev jobs, `evals/results/rag_variants.json`):

Variant	R@5 all	nDCG@10 all	R@5, 4+ reqs (n=368)	R@5 vague posts (n=15)	R@5 injected posts (n=48)
V0 naive dense	0.637	0.713	0.641	0.562	0.566
V2 hybrid	0.748	0.787	0.749	0.783	0.739
V3 hybrid + rerank	0.786	0.816	0.784	0.787	0.746
V8 corrective RAG	**0.798**	**0.828**	**0.796**	0.738	**0.777**

V1 0.716, V4 0.750 and V5 0.646 R@5 are in the same file. **Result:** on 4+ requirement jobs V8 beats V0 by 15.5 points, V2 by 10.8, so the hypothesis holds for V8, not for decomposition + hybrid alone. Two negative results: multi-vector (V5) barely beats V0, and on vague posts V8's Recall@5 is *below* V2/V3 (its nDCG is best: it ranks the one right item first and retrieves fewer of the others). By persona V8 R@5 is 0.785 / 0.820 / 0.790 (ML / full-stack / data).

Claim verification. Claims are split from the draft and graded against retrieved evidence with the cross-encoder; the supported/unsupported threshold was chosen by Youden's J on labelled pairs (-7.9 logit, AUC 0.787, `support_threshold.json`). On the Gemma V8 proposal run, 34% of drafted claims were unsupported before verification (27 of 79 claims, 1.35 per proposal) and 0.10 per proposal after (2 of 20 proposals kept one).

Downstream: decisions. Same 150 jobs, same model, threshold 0.60: V8 bid/skip F1 **0.950** (P 0.904, R 1.000) vs V0 **0.808** (P 0.967, R 0.694) (`decision_eval_open_v8_fixed_limit50.json`, `decision_eval_open_v0_fixed_limit50.json`). Naive retrieval does not make GigBid reckless, it makes it timid: missed evidence turns good jobs into skips. Price inside the fair band is 1.00 for both after the profile fix (section 8). **Downstream: proposals** (`proposal_eval_gemma_v8_gemma.json`, the first 10 gold-bid dev jobs per persona, `gemma` profile). Of 30 jobs, 20 were drafted and submitted to GigSim, 9 ended in `dead_letter` after provider errors (Google free-tier 429/500s) and 1 was put on watch. Simulated reply rate **0.30** (6 of 20), hires 0, Critic loops 1.6 on average, **8,369 tokens per proposal**, p95 878 s. The large model was often unavailable: the writer fell back from 31B to 26B 86 times and the Critic 45 times (`llm_fallbacks`), so most drafts were written by the smaller model. The LLM judge scored 10 of the 20 (the other

judge calls failed) with mean 3.9 of 5. V0 was not run at proposal level: the free-tier quota could not carry a third proposal run next to X2 and the injection run, so the V0 comparison rests on retrieval and decisions above. **Judge calibration** (`judge_calibration.json`): the author labelled 23 drafts blind (stratified by judge score, both X2 arms). Quadratic-weighted kappa **0.27** (fair), exact agreement 0.22, within one point 0.74; the judge is harsher (mean 3.57 vs 3.87). Six items differ by two or more points in both directions, so the bias is not a simple offset. The judge separated the X2 arms more than the author did (3.9 vs 3.3, author 4.0 vs 3.8). **Consequence:** judge scores are reported for completeness but no decision rests on them; X2 and the model comparison use the simulated client's reply probability and outcome from GigSim's client model (`gigsim/client_model.py`).

What ships. V8 (ADR 0004): best retrieval on every overall metric and on 4+ requirement and injected posts, p95 4.1 s vs V3's 7.5 s. The 1 GB live VM runs V2 because V8's cross-encoder overloaded it (measured: 20 cross-encoder pairs 1.5 s, load average 11); claim verification stays on there.

5. R&D question C: single agent vs prebuilt supervisor vs our graph

Setup (`evals/runners/agent_comparison.py`). Three designs on the same 17 non-duplicate dev jobs per persona (51), the same read-only MCP tools, the `gemma` profile and the same scoring (gold bid/skip, GigSim's client model for replies): a single ReAct agent with all tools, a prebuilt LangGraph supervisor with three ReAct workers, and our graph.

Design	Jobs	Provider errors	Bid/skip F1	Proposals	Unsupported claims / proposal	Reply rate	Tokens / job	p95
Single agent	51	50	not measurable	0	n/a	n/a	n/a	107 s
Prebuilt supervisor	51	51	not measurable	0	n/a	n/a	n/a	165 s
Our graph	51	**0**	**0.963** (P 1.00, R 0.93)	23	0.087	0.174	3,896	919 s

Source: `agent_comparison_graph.json` and its rows. **Result.** The comparison of answer quality is inconclusive: the two baselines failed on 50 and 51 of 51 jobs (429 quota 78, 500/503 17, timeouts 6) while the graph failed on none, in the same run on the same key. The difference is structural, not luck: the graph routes each node to its own model, paces calls and falls back from 31B to 26B (`llm_fallbacks`: 117 fallbacks in this run), while a prebuilt ReAct agent has one model, no fallback and fails the whole task on any provider error. On a free-tier provider that is the dominant property. The graph also kept 9 of 11 scam or injected jobs out of bidding and had no policy violations. **Rerun** (`agent_comparison_baselines.json`, merged on shared jobs in `agent_comparison_matched.json`). To rule out quota contention, the baselines were rerun alone on 26B with 10 retries and pacing, on the first 6 of the same jobs per persona (18). They still failed on 12 (single: 500s 6, 429s 4, timeouts 2) and 16 (supervisor: timeouts 10, 500s 6) of 18. The 6 answers the single agent did give were all correct skips; the supervisor answered 2, skipping a gold bid. On the same 18 jobs from its own run, the graph had 0 errors, F1 1.00 and 9 proposals (reply rate 0.22). The arms ran at different times, so this is not a strictly paired comparison, and the baselines' answer quality is still not measured; what is measured, twice, is that single-model prebuilt agents do not survive this provider.

6. The change request

What changed. No official policy text was published for the class (checked 12 Nov), so we defined one, modelled on real marketplace changes, and absorbed it exactly as we would an official one (ADR 0007): Connects cost x1.5, a flat 2-Connect fee per proposal, at most 10 submissions a day (was 30), cover letters at most 250 words.

What we had to edit (commit d988242, about 10 minutes including the test): the platform side, `gigsim/policy.yaml` (+14/-8); one config line, `policy.expected_version: v2`; and **one GigBid code change, 3 lines** in `agent/nodes/pricing.py`. The multiplier, the word cap and the policy version flowed through `GET /policy` with no edit. A new integration test switches the simulator to v2 and checks both pricing and compliance.

What it says about the design. Keeping the numbers in configuration worked; re-implementing the platform's *formula* did not. Pricing knew `connects_multiplier` but not `connects_flat_fee`, so without the fix it quoted 14 Connects for a proposal the platform charged 16, and both the expected-value and the balance checks would have been wrong. The test fails on the old code with exactly that (`assert 14 == 16`). The sturdier design is to ask the platform for the cost of this proposal (the draft endpoint already returns it in `dry_run`) rather than recompute it. Effect on bidding (`change_request_v2_impact.json`): Connects per proposal 8.1 -> 14.3, no bid becomes negative expected

value, and the same 84 dev bids need 1,198 Connects instead of 684 and nine days instead of three at 10 a day. The daily cap is enforced only by the platform; GigBid does not yet plan approvals across days.

7. Signature extension: X2 proposal A/B test

Arm A evidence-led (shipped writer) vs arm B plan-first (`agent/prompts/writer_plan.md`), same jobs, model, retrieval and simulator; hypothesis registered in `docs/EXTENSION.md` before arm B ran (commit d7d6f94). | | Arm A evidence-led | Arm B plan-first | |---|---|---| | Drafted / 30 jobs | 20 | 21 | | Unsupported claims per proposal, before / after verification | 1.35 / 0.10 | **5.14** / 0.24 | | LLM judge (judged drafts) | 3.9 (10) | 3.3 (13) | | Reply rate, 15 paired jobs | 0.20 | 0.13 | | Mean simulated reply probability, paired | 0.340 | 0.336 | | Tokens per proposal | 8,369 | 9,671 |

Sources: `ab_test_x2.json`, `proposal_eval_gemma_v8_gemma.json`, `proposal_eval_gemma_planfirst_v8_gemma.json`.

Result. No detectable difference: paired difference in reply probability B - A = -0.004 (bootstrap 95% CI -0.091 to 0.095, Wilcoxon p = 0.72); McNemar on replies p = 1.0 (2 pairs only A replied, 1 only B). The pre-registered prediction (B not better) holds, but with 15 pairs the minimum detectable reply-rate difference is 0.38, so this is "no large effect", not "no effect". The clearer finding is a side effect: plan-first drafts made **almost four times as many unsupported claims** before verification (73% of claims vs 34%), because a delivery plan invites statements about the freelancer's process that the portfolio does not record. The verifier removed most of them, at the cost of more Critic loops (1.95 vs 1.6). **Deviations from the registration:** the `gemma` profile instead of `open` (Cerebras credit ran out), 15 pairs instead of about 30 (provider dead-letters in both arms), and the writer fell back from 31B to 26B in both arms (86 and 88 times), so both arms were largely written by the same smaller model.

8. Failure analysis

The first V8 decision eval (`decision_eval_open_v8_limit50_rows.jsonl`, 150 jobs) scored F1 0.900 but had **19 wrong decisions, all bids on gold-skip jobs**, and 32 of the correct bids priced outside the fair band (28 below it).

Category	Jobs (examples)	Cause	Fix tried	Worked?
Wrong bid: budget below floor	5 (j-00003 $24/h vs $30 floor; j-00044 $160 vs $300 minimum)	`profile` was not declared in `BidState`, so LangGraph dropped it and `decide` saw a floor of 0	declare `profile` in the state schema; regression test that fails on the old schema (commit f9badfa)	**yes**: 0 of these left
Mispriced, below band	28 (j-00008, j-00015, j-00023 bid $28/h, band from $39)	same bug: pricing used its default $20 floor x 1.4	same fix	**yes**: price in band 0.73 -> 1.00
Wrong bid: over-generous evidence	9 (j-00005, j-00027, j-00206, j-00227, j-00245: gold coverage 0.4, fit 0.67)	retrieval finds loosely related evidence for missing skills (Next.js, GraphQL, Unity, dbt)	bid threshold 0.60 -> 0.70 from a dev-set sweep (`decision_threshold_sweep.json`)	**partly**: fixes 8 of 9, loses 2 true bids; F1 0.950 -> 0.982 in-sample only. The real fix (stricter support on the fit side) is not done
Missed evidence	26 jobs wrong with V0 and right with V8 (pre-fix runs; 8 the other way)	dense retrieval misses paraphrased requirements	V8 corrective RAG	**yes**: recall 0.694 -> 1.000 after the fix

After the profile fix the measured V8 run has 9 errors, all in the over-generous-evidence category, F1 0.950, accuracy 0.94 (`decision_eval_open_v8_fixed_limit50.json`). The bug was invisible in the headline metric (0.900 looks good) and in unit tests (the `decide` function was correct); only reading individual failures found it. **Proposal level** (`proposal_eval_gemma_v8_gemma_rows.jsonl`, 20 submitted proposals). *Missing must-haves:* the simulated client's main reason for not replying was must-have coverage in the text: replied proposals covered 0.92 of the job's must-haves on average, lost ones 0.74, and 8 of 20 covered fewer than all. Drafts are short (43 to 130 words, median about 94), so the writer drops requirements the Fit Matcher had found evidence for; the fix is to require the writer to mention every must-have the Fit Matcher supported (not done; neither X2 arm enforces it). *Unsupported claims:* 27 of 79, all but 2

removed by the verifier before submission. *Tool and provider errors:* 9 of 30 runs dead-lettered on Google 429/500s after retries, which the per-node retry and dead-letter design handled (no crash, no partial submission); the fix is pacing (`LLM_PACE`), applied to later runs. No compliance blocks, no injection echoes in these 20.

9. Safety and platform respect

GigBid must never submit, send or spend without approval; never contact clients off-platform; never fabricate experience; never follow instructions inside job posts or messages; never scrape. Enforcement: interrupts on every write, idempotency keys, per-agent tool allowlists, claim verification + Critic + Compliance Guard, untrusted-text delimiting and quarantine (`agent/guardrails.py`), rate limits far below human use in live mode (RUNBOOK). **What still gets through.** Detector level (`injection_eval_gemma_detector.json`): the regex detector flags all 71 planted dev-set injections with 0 false positives on 529 clean posts, and **0 of 12 paraphrased attacks** (canary, off-platform contact, prompt leak, price manipulation). The detector only catches phrasings it was written for, so it is a tripwire, not the defence. What has to hold is the next layer: posts are delimited as untrusted data and every prompt says to treat them as data, plus output checks (canary echo, contact details, below-floor price) in the Compliance Guard.
Deviation: the end-to-end run, with and without those defences, timed out twice on the free tier (288 trials at 350 minutes, then a rerun on 72); the end-to-end attack success rate is therefore not reported. Beyond the detector numbers above, the defence is a design claim, not a measured one.

10. Results against declared budgets

The hidden test set runs after the deadline; these are dev-set measurements against `agent/config.yaml: budgets`.

Budget	Declared	Measured	Source	Met?
p95 latency, job analysed	25 s	12.8 s (Cerebras `open`)	`decision_eval_open_v8_fixed_limit50.json`	yes
p95 latency, proposal	90 s	878 s (Gemma free tier)	`proposal_eval_gemma_v8_gemma.json`	**no**
Tokens per job	60,000 max	2,741 per job analysed, 8,369 per proposal	same files	yes
Cost per job / proposal	USD 0.004 / 0.02	USD 5.27 for all 11 Nov Cerebras runs (promo credit); USD 0 on Gemma	Cerebras billing, section 1	not separable per run
Critic revisions	2 max	1.6 loops on average, never above the cap	proposal eval	yes

The proposal latency miss is the provider, not the graph: Gemma 4 always emits hidden reasoning, the free tier returns frequent 429/500s, and each retry waits. Proposal p50 was 479 s. Meeting 90 s needs a paid or faster endpoint; the 12.8 s decision p95 on Cerebras shows the graph itself is not the bottleneck.